DSA Track

Interview Preparation Series • Professional Edition

Data Structures

02

Compiled with LaTeX • June 15, 2026

harsha.somisetty01@gmail.com

In This Edition

> Dense reference: mechanics, complexities, code, diagrams, and interview takeaways in one place.

1 Overview

Data structures are the substrate of every algorithm. Choosing the wrong one is the single most common reason candidates fail coding interviews — the algorithm is right, but O(n) lookups in a list where a hash map would give O(1) makes the solution too slow. This file is organized as a working reference: read top-to-bottom once, then use the complexity table and decision guide during revision.

Mental model for selection:

1. What operations do I need? (search, insert, delete, min/max, range, prefix?)
2. What are the frequency/order constraints? (random access? FIFO? LIFO? sorted?)
3. What are the acceptable complexities?
4. Then pick the structure.

2 Master Complexity Table

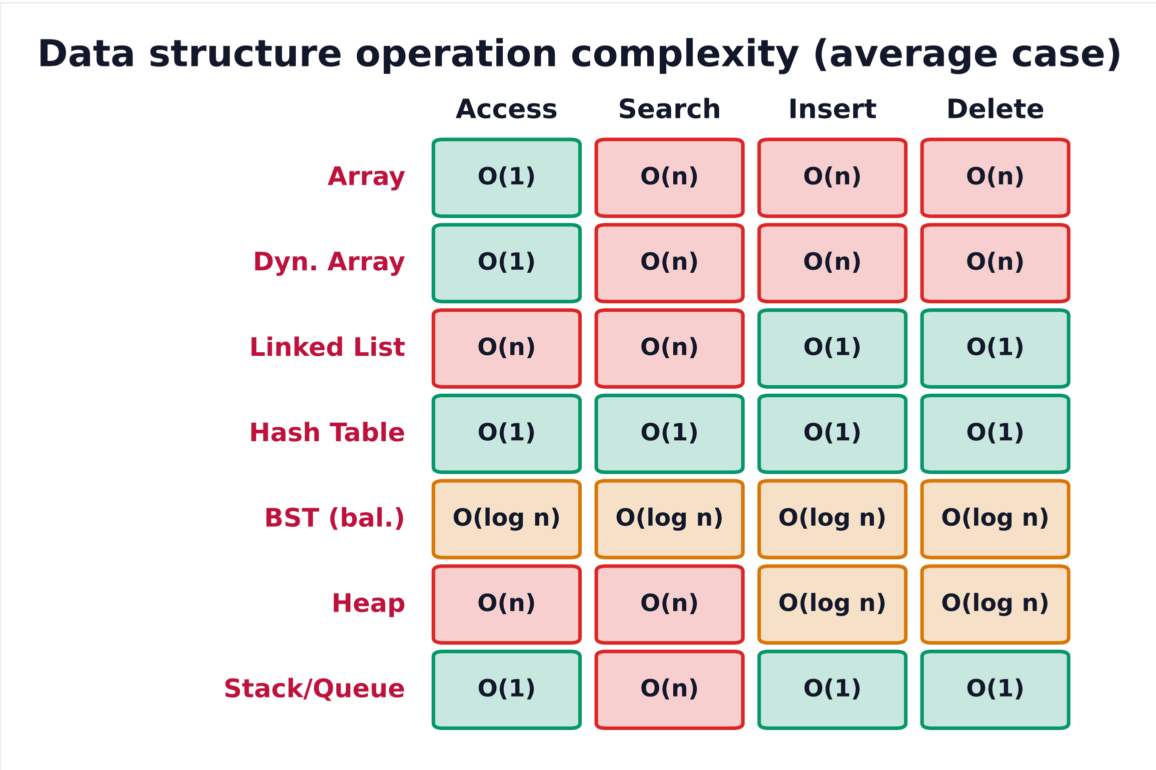

Figure 1. Pick the structure that makes your hot operation O(1)/O(log n). Hash tables win for lookups; balanced trees keep order; heaps give cheap min/max.

All complexities are average-case unless marked. W = worst case.

Data Structure	Access	Search	Insert	Delete	Space	Notes
Array (static)	O(1)	O(n)	O(n)	O(n)	O(n)	Insert/delete shifts elements

Data Structure	Access	Search	Insert	Delete	Space	Notes
Dynamic Array (list)	O(1)	O(n)	O(1) amort	O(n)	O(n)	Append amortized O(1)
String (immutable)	O(1)	O(n)	O(n)	O(n)	O(n)	Concat is O(n); use StringBuilder
Hash Table	N/A	O(1)	O(1)	O(1)	O(n)	W: O(n) with all collisions
Singly Linked List	O(n)	O(n)	O(1) head	O(n)	O(n)	O(1) if you have the node reference
Doubly Linked List	O(n)	O(n)	O(1) ends	O(1)*	O(n)	*O(1) if node ref known
Stack	O(n)	O(n)	O(1) push	O(1) pop	O(n)	Top access O(1)
Queue	O(n)	O(n)	O(1) enq	O(1) deq	O(n)	Front access O(1)
Deque	O(1) ends	O(n)	O(1) ends	O(1) ends	O(n)	Both ends O(1)
BST (balanced)	O(log n)	O(log n)	O(log n)	O(log n)	O(n)	Unbalanced: O(n) worst
BST (unbalanced)	O(n) W	O(n) W	O(n) W	O(n) W	O(n)	Degenerate = linked list
AVL Tree	O(log n)	O(log n)	O(log n)	O(log n)	O(n)	Strict balance; fast lookup
Red-Black Tree	O(log n)	O(log n)	O(log n)	O(log n)	O(n)	Looser balance; fast insert/delete
Binary Heap (min/max)	O(1) top	O(n)	O(log n)	O(log n)	O(n)	Build heap: O(n)
Trie	O(m)	O(m)	O(m)	O(m)	O(n*m)	m = key length
Graph (adj list)	O(V+E)	O(V+E)	O(1)	O(E)	O(V+E	Sparse graphs
Graph (adj matrix)	O(1)	O(1) edge	O(1)	O(1)	O(V^2)	Dense graphs
Union-Find	$O(\alpha(n))$	$O(\alpha(n))$	$O(\alpha(n))$	-	O(n)	α = inverse Ackermann, $\approx$ O(1)
Segment Tree	O(log n)	O(log n)	O(log n)	O(log n)	O(n)	Range queries + point updates
Fenwick Tree (BIT)	O(log n)	O(log n)	O(log n)	O(log n)	O(n)	Simpler than segment tree
LRU Cache	O(1)	O(1)	O(1)	O(1)	O(n)	Hash map + doubly linked list
Bloom Filter	N/A	O(k)	O(k)	N/A	O(m)	k=hash funcs, probabilistic

3 The Data Structures

3.1 Arrays & Dynamic Arrays

What it is: Contiguous block of memory where each element sits at a fixed offset from the base address. Random access is O(1) because address = base + index * element_size.

Internal mechanics:

```
Static Array (size=6):
Index:  [0]  [1]  [2]  [3]  [4]  [5]
Value:  [10] [20] [30] [40] [50] [60]
         ^                         ^
```

```
        base addr                 base + 5*sizeof(int)

Dynamic Array (Python list) — amortized resize:
size=4, capacity=4:   [1][2][3][4][ ][ ][ ][ ]  ← capacity doubles on overflow
Append 5 → capacity=8:[1][2][3][4][5][ ][ ][ ]
Append cost analysis (n appends):
  - n-1 cheap appends: O(1) each
  - ~log(n) doublings: cost 1+2+4+...+n = 2n copies total
  - Amortized per append: 2n / n = O(1) ✓
```

Complexity:

Op	Best	Avg	Worst
Access	O(1)	O(1)	O(1)
Search	O(1)	O(n)	O(n)
Append	O(1)	O(1)	O(n)
Insert	O(1)	O(n)	O(n)
Delete	O(1)	O(n)	O(n)

When to use:

- Random access by index is needed
- Data is mostly read; writes are appends
- Cache locality matters (iteration speed)

When NOT to use:

- Frequent mid-list insertions/deletions (use linked list or deque)
- Need fast search (use hash map or BST)

Key Python patterns:

```python
# Two-pointer on sorted array
left, right = 0, len(arr) - 1
while left < right:
    s = arr[left] + arr[right]
    if s == target: return [left, right]
    elif s < target: left += 1
    else: right -= 1

# Sliding window
window_sum = sum(arr[:k])
max_sum = window_sum
for i in range(k, len(arr)):
    window_sum += arr[i] - arr[i - k]
    max_sum = max(max_sum, window_sum)

# In-place reversal
arr[i], arr[j] = arr[j], arr[i]
```

Interview takeaway: Arrays are the default. When the problem says "return sorted" or "in-place", think array + two pointers. When you need to insert/delete mid-sequence often, question whether an array is right.

Real interview use-case: Two Sum (hash map), Sliding Window Maximum, Kadane's (max subarray), rotate array in-place, merge sorted arrays.

3.2 Strings

What it is: Sequence of characters. In Python and Java, strings are **immutable** — every concatenation creates a new object. In C++, `std::string` is mutable but each `+=` may reallocate.

Internal mechanics:

```
Python string "hello":
  h | e | l | l | o
  Each character is a Unicode code point (Python 3: variable width internally)
  id("hello" + "x") ≠ id("hello")  ← new object every concat

String concatenation cost in a loop (naïve):
  for i in range(n): result += s[i]
  Iteration 1: copy 1 char    → O(1)
  Iteration 2: copy 2 chars   → O(2)
  ...
  Iteration n: copy n chars   → O(n)
  Total: O(1+2+...+n) = O(n²)  ← DISASTER for large n

String Builder pattern:
  parts = []
  for c in chars: parts.append(c)   ← O(1) amortized each
  result = "".join(parts)            ← O(n) one pass
  Total: O(n)  ✓
```

Complexity:

Op	Complexity	Note
Index access	O(1)	
Slice s[i:j]	O(j-i)	Creates new string
Concatenate	O(n+m)	Both strings copied
Find/search	O(n*m)	Naive; KMP/Z gives O(n+m)
Comparison	O(n)	Character by character
Join	O(n)	One pass

Key Python patterns:

Python

```python
# Always join, never += in loop
result = "".join(list_of_chars)

# Reverse a string
rev = s[::-1]

# Check palindrome
def is_palindrome(s):
    s = s.lower()
    l, r = 0, len(s) - 1
    while l < r:
        while l < r and not s[l].isalnum(): l += 1
        while l < r and not s[r].isalnum(): r -= 1
        if s[l] != s[r]: return False
        l += 1; r -= 1
    return True

# Anagram check
from collections import Counter
Counter(s) == Counter(t)
```

```python
# Sliding window on string (distinct chars)
from collections import defaultdict
freq = defaultdict(int)
l = 0
for r, c in enumerate(s):
    freq[c] += 1
    while len(freq) > k:
        freq[s[l]] -= 1
        if freq[s[l]] == 0: del freq[s[l]]
        l += 1
```

Interview takeaway: Never concatenate strings in a loop. Use `"".join(parts)`. For pattern matching, think sliding window + frequency map. Anagram/permutation problems → character frequency counter.

Real interview use-case: Longest substring without repeating characters, group anagrams, valid palindrome, minimum window substring, encode/decode strings.

3.3 Hash Tables

What it is: Maps keys to values in O(1) average by computing `hash(key) % capacity` to find a bucket, then storing the key-value pair there.

Internal mechanics:

```
Hash Table with Chaining (load factor = 0.75):

  Buckets:
  [0] → None
  [1] → ("apple", 5) → ("grape", 2) → None   ← collision! same bucket
  [2] → ("banana", 3) → None
  [3] → None
  [4] → ("cherry", 7) → None
  ...

  hash("apple")  % 8 = 1
  hash("grape")  % 8 = 1   ← collision → chain extends
  hash("banana") % 8 = 2

Open Addressing (linear probing):
  [0]: empty
  [1]: ("apple", 5)
  [2]: ("grape", 2)   ← collision at 1, probe to 2
  [3]: ("banana", 3)
  ...

Load Factor (α) = n / capacity
  α < 0.7: few collisions, O(1) expected
  α > 0.9: many collisions → O(n) worst
  → resize (rehash all) when α threshold exceeded
  Python dict resizes at 2/3 full → rehash to 2x capacity

Good Hash Function Properties:
  1. Deterministic (same key → same hash)
  2. Uniform distribution (minimize collisions)
  3. Fast to compute
  4. Avalanche effect (small key change → big hash change)
```

Collision resolution comparison:

Strategy	Pro	Con
Chaining	Simple, works at high load	Extra memory for pointers
Linear probing	Cache-friendly	Clustering degrades performance

Strategy	Pro	Con
Quadratic probe	Less clustering	May not probe all slots
Double hashing	Best distribution	Two hash functions needed

Complexity:

Op	Average	Worst
Search	O(1)	O(n)
Insert	O(1)	O(n)
Delete	O(1)	O(n)

When to use:

- O(1) lookup by key
- Counting frequencies
- Caching / memoization
- Deduplication

When NOT to use:

- Need sorted order (use BST/sorted array)
- Keys not hashable (use tree map)
- Memory is very tight

Key Python patterns:

Python

```python
from collections import defaultdict, Counter

# Frequency count
freq = Counter("anagram")          # Counter({'a': 3, 'n': 1, ...})

# Default dict avoids KeyError
graph = defaultdict(list)
graph['A'].append('B')

# Two Sum O(n)
def two_sum(nums, target):
    seen = {}
    for i, n in enumerate(nums):
        complement = target - n
        if complement in seen:
            return [seen[complement], i]
        seen[n] = i

# Grouping anagrams
from collections import defaultdict
def group_anagrams(strs):
    groups = defaultdict(list)
    for s in strs:
        groups[tuple(sorted(s))].append(s)
    return list(groups.values())
```

Interview takeaway: Hash map is your first tool when you need O(1) lookup. "Two pointer O(n^2)? Can we trade space for time?" → hash map. Watch out: dict keys must be hashable (no lists as keys; use tuples).

Real interview use-case: Two Sum, subarray sum equals k, longest consecutive sequence, top K frequent elements, valid sudoku.

3.4 Linked Lists

What it is: Sequence of nodes where each node holds a value and a pointer to the next (singly) or both next and previous (doubly) nodes. Non-contiguous memory.

Internal mechanics:

```
Singly Linked List:
  head
   ↓
  [1|•]→[2|•]→[3|•]→[4|•]→None
   node   next pointer

Doubly Linked List:
  head                              tail
   ↓                                 ↓
None←[•|1|•]⇔[•|2|•]⇔[•|3|•]⇔[•|4|•]→None
     prev val next

Insert at head (O(1)):
  new_node.next = head
  head = new_node

Insert at tail (O(1) with tail pointer):
  tail.next = new_node
  tail = new_node

Delete node (given reference, doubly linked, O(1)):
  node.prev.next = node.next
  node.next.prev = node.prev

Delete node (singly linked, given reference only, O(n)):
  Must traverse from head to find predecessor
  → trick: copy next node's value into current, delete next
```

Complexity:

Op	Singly LL	Doubly LL
Access by index	O(n)	O(n)
Search	O(n)	O(n)
Insert at head	O(1)	O(1)
Insert at tail	O(n)/O(1)	O(1)
Insert at middle	O(n)	O(n)
Delete at head	O(1)	O(1)
Delete by ref	O(n)	O(1)

When to use:

- Frequent insertions/deletions at head or known positions
- Implementing stacks, queues, LRU cache
- When you don't need random access

When NOT to use:

- Random access needed (use array)
- Memory is tight (each node has pointer overhead)
- Cache performance critical (non-contiguous = cache misses)

Key Python patterns:

```python
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

# Reverse a linked list (iterative)
def reverse(head):
    prev = None
    curr = head
    while curr:
        nxt = curr.next
        curr.next = prev
        prev = curr
        curr = nxt
    return prev

# Floyd's cycle detection
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow is fast:
            return True
    return False

# Find middle (slow/fast pointers)
def find_middle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow

# Dummy node pattern (simplifies edge cases)
def delete_nth_from_end(head, n):
    dummy = ListNode(0, head)
    fast = slow = dummy
    for _ in range(n + 1):
        fast = fast.next
    while fast:
        fast = fast.next
        slow = slow.next
    slow.next = slow.next.next
    return dummy.next
```

Interview takeaway: Always use a dummy head node to avoid null checks at head. Slow/fast pointer solves cycle detection, middle finding, and nth-from-end in one pass. Linked list problems are usually just pointer manipulation — draw it out.

Real interview use-case: Reverse linked list, merge two sorted lists, detect cycle, reorder list, LRU cache.

3.5 Stacks & Queues

What it is:

- **Stack**: LIFO (Last In, First Out). Like a stack of plates.
- **Queue**: FIFO (First In, First Out). Like a line at a store.
- **Deque**: Double-ended queue; insert/delete at both ends O(1).

- **Monotonic stack/queue**: Stack/queue that maintains monotone order for range queries.
- **Circular buffer**: Fixed-size ring buffer for queue; avoids shifting.

Internal mechanics:

```
Stack (LIFO):
  Push 1,2,3:  [1]→[2]→[3]   ← top is 3
  Pop:         [1]→[2]        ← returns 3

Queue (FIFO) using circular buffer:
  Capacity=5, head=0, tail=0
  Enqueue 1,2,3:
    [1][2][3][ ][ ]   head=0, tail=3
  Dequeue:
    [ ][2][3][ ][ ]   head=1, tail=3
  Enqueue 4,5:
    [ ][2][3][4][5]   head=1, tail=0 (wrapped!)
  → indices mod capacity: (tail+1) % cap

Monotonic Stack (next greater element):
  arr = [2, 1, 5, 3, 4]
  Stack tracks indices of elements waiting for their "next greater"
  i=0: push 0       stack=[0]         (arr[0]=2)
  i=1: 1<2, push 1  stack=[0,1]       (arr[1]=1)
  i=2: 5>1, pop 1→ans[1]=5, 5>2 pop 0→ans[0]=5, push 2
                    stack=[2]
  i=3: 3<5, push 3  stack=[2,3]
  i=4: 4>3, pop 3→ans[3]=4, 4<5 push 4
                    stack=[2,4]
  Remaining: no greater element → -1
  ans = [5, 5, -1, 4, -1]

Deque (collections.deque in Python):
  appendleft / popleft  → O(1)
  append / pop          → O(1)
```

Complexity:

Op	Stack	Queue	Deque	Monotonic Stack
Push/Enqueue	O(1)	O(1)	O(1)	O(1) amort
Pop/Dequeue	O(1)	O(1)	O(1)	O(1) amort
Peek (top/front)	O(1)	O(1)	O(1)	O(1)
Search	O(n)	O(n)	O(n)	O(n)

When to use:

- Stack: DFS, expression parsing, undo operations, backtracking
- Queue: BFS, task scheduling, sliding window with deque
- Monotonic stack: next greater/smaller element, largest rectangle in histogram
- Deque: sliding window maximum, palindrome check

Key Python patterns:

Python

```python
from collections import deque

# Stack (use list, pop from end)
stack = []
stack.append(x)     # push
stack.pop()         # pop O(1)
stack[-1]           # peek
```

```python

# Queue (use deque for O(1) dequeue)
q = deque()
q.append(x)        # enqueue
q.popleft()        # dequeue O(1)

# BFS template
def bfs(root):
    q = deque([root])
    while q:
        node = q.popleft()
        for child in node.children:
            q.append(child)

# Monotonic stack — next greater element
def next_greater(nums):
    n = len(nums)
    ans = [-1] * n
    stack = []  # indices
    for i, num in enumerate(nums):
        while stack and nums[stack[-1]] < num:
            idx = stack.pop()
            ans[idx] = num
        stack.append(i)
    return ans

# Sliding window maximum (monotonic deque)
def sliding_max(nums, k):
    dq = deque()   # indices, decreasing values
    res = []
    for i, num in enumerate(nums):
        while dq and nums[dq[-1]] < num:
            dq.pop()
        dq.append(i)
        if dq[0] == i - k:  # out of window
            dq.popleft()
        if i >= k - 1:
            res.append(nums[dq[0]])
    return res
```

Interview takeaway: Monotonic stack is the pattern for "next greater/smaller element" and "largest rectangle" problems. BFS always uses a queue. For sliding window max/min, use a monotonic deque. Never use `list.pop(0)` for queues — it's O(n); use `deque.popleft()`.

Real interview use-case: Valid parentheses, min stack, daily temperatures, largest rectangle in histogram, sliding window maximum.

3.6 Trees

What it is: Hierarchical, acyclic connected graph. Root at top, leaves at bottom.

Internal mechanics:

```
Binary Tree (each node: up to 2 children):
           1
          / \
         2   3
        / \   \
       4   5   6

BST (Binary Search Tree): left < root < right
           8
```

```
         / \
        3   10
       / \    \
      1   6   14
         / \  /
        4   7 13

BST invariant: for any node N:
  - All nodes in left subtree < N.val
  - All nodes in right subtree > N.val
  → In-order traversal gives sorted sequence!

BST Search:
  Find 6: start at 8 → 6<8 go left → 3 → 6>3 go right → 6 ✓
  O(h) where h = height. Balanced: h=log n. Skewed: h=n.

Balanced Tree (AVL) — height constraint:
  Balance factor = height(left) - height(right)
  Must be in {-1, 0, 1} for every node
  Rotation on violation:
    Left-Left: right rotate
    Right-Right: left rotate
    Left-Right: left rotate then right rotate
    Right-Left: right rotate then left rotate

Red-Black Tree (less strict, used in Java TreeMap):
  Properties:
  1. Every node is red or black
  2. Root is black
  3. Red nodes have black children
  4. All paths from node to null have same # black nodes
  → Height ≤ 2*log(n+1) guaranteed

Tree Traversals:
  Pre-order:   root → left → right   [1,2,4,5,3,6]
  In-order:    left → root → right   [4,2,5,1,3,6] ← sorted for BST!
  Post-order: left → right → root    [4,5,2,6,3,1]
  Level-order: BFS level by level    [1,2,3,4,5,6]
```

Complexity:

Op	Balanced BST	Unbalanced BST	AVL / RB Tree
Search	O(log n)	O(n)	O(log n)
Insert	O(log n)	O(n)	O(log n)
Delete	O(log n)	O(n)	O(log n)
Min/Max	O(log n)	O(n)	O(log n)
In-order	O(n)	O(n)	O(n)

When to use:

- BST: sorted data with dynamic insertions/deletions
- AVL/RB: guaranteed O(log n) when data can be adversarial
- Any tree: hierarchical relationships, recursive decomposition

Key Python patterns:

```python
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
```

```python

# In-order traversal (recursive)
def inorder(root):
    if not root: return []
    return inorder(root.left) + [root.val] + inorder(root.right)

# In-order traversal (iterative — important for interviews)
def inorder_iter(root):
    result, stack = [], []
    curr = root
    while curr or stack:
        while curr:
            stack.append(curr)
            curr = curr.left
        curr = stack.pop()
        result.append(curr.val)
        curr = curr.right
    return result

# Height of tree
def height(root):
    if not root: return 0
    return 1 + max(height(root.left), height(root.right))

# Check if BST is valid
def is_valid_bst(root, lo=float('-inf'), hi=float('inf')):
    if not root: return True
    if not (lo < root.val < hi): return False
    return (is_valid_bst(root.left, lo, root.val) and
            is_valid_bst(root.right, root.val, hi))

# LCA (Lowest Common Ancestor) in BST
def lca_bst(root, p, q):
    if p.val < root.val and q.val < root.val:
        return lca_bst(root.left, p, q)
    if p.val > root.val and q.val > root.val:
        return lca_bst(root.right, p, q)
    return root

# Level order traversal
from collections import deque
def level_order(root):
    if not root: return []
    result, q = [], deque([root])
    while q:
        level = []
        for _ in range(len(q)):
            node = q.popleft()
            level.append(node.val)
            if node.left: q.append(node.left)
            if node.right: q.append(node.right)
        result.append(level)
    return result
```

Interview takeaway: BST in-order = sorted. Use recursion with min/max bounds for BST validation, not just left < root < right (common bug). Diameter of tree = longest path through any node = max(left_depth + right_depth) over all nodes.

Real interview use-case: Validate BST, lowest common ancestor, diameter of binary tree, serialize/deserialize tree, path sum, kth smallest in BST.

3.7 Heaps / Priority Queues

What it is: A complete binary tree stored as an array, where each parent satisfies the heap property (min-heap: parent $\leq$ children; max-heap: parent $\geq$ children). Top element is always the min (or max).

Internal mechanics:

```
Min-Heap (array representation):
         1
        / \
       3   5
      / \ / \
     7  4 8  9

Array: [1, 3, 5, 7, 4, 8, 9]
Index:  0  1  2  3  4  5  6

Index relationships:
  Parent(i)      = (i-1) // 2
  Left child(i)  = 2*i + 1
  Right child(i) = 2*i + 2

Sift Up (after insert at end):
  Insert 2: append → [1,3,5,7,4,8,9,2]
  Compare 2 with parent (index 3): arr[3]=7 > 2 → swap
  → [1,3,5,2,4,8,9,7]
  Compare 2 with parent (index 1): arr[1]=3 > 2 → swap
  → [1,2,5,3,4,8,9,7]
  Compare 2 with parent (index 0): arr[0]=1 < 2 → stop ✓

Sift Down (after extracting root — put last element at root):
  Extract 1: put 7 at root → [7,2,5,3,4,8,9]
  Compare 7 with children 2,5: min child=2 → swap
  → [2,7,5,3,4,8,9]
  Compare 7 with children 3,4: min child=3 → swap
  → [2,3,5,7,4,8,9]
  7 has no children → stop ✓

Build Heap (heapify) from arbitrary array — O(n):
  Start sifting down from last non-leaf (n//2 - 1) to root
  Counterintuitive: O(n) not O(n log n) because most nodes are near leaves
```

Complexity:

Op	Time
Get min/max	O(1)
Insert	O(log n)
Extract min	O(log n)
Build heap	O(n)
Search	O(n)
Heapsort	O(n log n)

When to use:

- Get the min/max repeatedly (priority queues)
- Top K elements (use heap of size K)
- Merge K sorted lists
- Dijkstra's shortest path
- Median maintenance (two heaps)

Key Python patterns:

Python

```python
import heapq

# Min-heap (default)
heap = []
heapq.heappush(heap, 3)
heapq.heappush(heap, 1)
heapq.heappush(heap, 2)
heapq.heappop(heap)    # returns 1

# Max-heap: negate values
max_heap = []
heapq.heappush(max_heap, -3)
heapq.heappush(max_heap, -1)
-heapq.heappop(max_heap)   # returns 3

# Build heap in O(n)
nums = [3, 1, 4, 1, 5, 9]
heapq.heapify(nums)

# Top K elements
def top_k(nums, k):
    return heapq.nlargest(k, nums)   # O(n log k)

# Top K using min-heap of size k (space efficient)
def top_k_heap(nums, k):
    heap = []
    for n in nums:
        heapq.heappush(heap, n)
        if len(heap) > k:
            heapq.heappop(heap)
    return sorted(heap, reverse=True)

# Merge K sorted lists
def merge_k_lists(lists):
    heap = []
    for i, lst in enumerate(lists):
        if lst:
            heapq.heappush(heap, (lst[0], i, 0))
    result = []
    while heap:
        val, i, j = heapq.heappop(heap)
        result.append(val)
        if j + 1 < len(lists[i]):
            heapq.heappush(heap, (lists[i][j+1], i, j+1))
    return result

# Median maintenance (two heaps)
class MedianFinder:
    def __init__(self):
        self.lo = []   # max-heap (negate)
        self.hi = []   # min-heap
    def add_num(self, num):
        heapq.heappush(self.lo, -num)
        heapq.heappush(self.hi, -heapq.heappop(self.lo))
        if len(self.hi) > len(self.lo):
            heapq.heappush(self.lo, -heapq.heappop(self.hi))
    def find_median(self):
        if len(self.lo) > len(self.hi): return -self.lo[0]
        return (-self.lo[0] + self.hi[0]) / 2.0
```

Interview takeaway: Python only has min-heap via heapq; negate values for max-heap.

`heapify` is O(n). Top K smallest: use max-heap of size K and push out the max when size exceeds K. Top K largest: use min-heap of size K.

Real interview use-case: Kth largest element, merge K sorted lists, task scheduler, find median from data stream, Dijkstra's.

3.8 Tries (Prefix Trees)

What it is: A tree where each path from root to leaf spells a word, and each node represents one character. Shared prefixes share nodes.

Internal mechanics:

```
Trie storing: ["cat", "car", "card", "care", "bat"]

          root
         /    \
        c      b
        |      |
        a      a
       / \      \
      t   r      t
     (*)  |      (*)
         / \
        d   e
       (*)  (*)

(*) = is_end marker

Node structure:
  children: dict[char → TrieNode]  (or array[26])
  is_end: bool

Insert "card":
  root → c (create if absent)
       → a (create if absent)
       → r (exists, reuse)
       → d (create)
       mark d.is_end = True

Search "care":
  root → c → a → r → e → is_end? True ✓

StartsWith "ca":
  root → c → a → exists? True ✓ (prefix search)

Space: each node uses O(ALPHA) where ALPHA=26 for lowercase
  n strings of avg length m: O(n*m) nodes worst case
  (better than storing all strings if many shared prefixes)
```

Complexity (m = length of key):

Op	Time	Space
Insert	O(m)	O(m)
Search	O(m)	O(1)
StartsWith	O(m)	O(1)
Delete	O(m)	O(1)

When to use:

- Autocomplete / prefix search
- Spell checking
- IP routing tables
- Word search in a grid

When NOT to use:

- Simple key-value lookup (use hash map)
- Keys are not strings or have no shared prefixes

Key Python patterns:

```python
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word: str) -> None:
        node = self.root
        for ch in word:
            if ch not in node.children:
                node.children[ch] = TrieNode()
            node = node.children[ch]
        node.is_end = True

    def search(self, word: str) -> bool:
        node = self.root
        for ch in word:
            if ch not in node.children:
                return False
            node = node.children[ch]
        return node.is_end

    def starts_with(self, prefix: str) -> bool:
        node = self.root
        for ch in prefix:
            if ch not in node.children:
                return False
            node = node.children[ch]
        return True

# Word search II (trie + DFS backtracking)
# Build trie from word list, then DFS on board
# When we reach a trie end node, we found a word
```

Interview takeaway: Trie is the answer to "find all words with prefix X" or "word search on a grid with a word list". For word search II (board + word list), build a trie from words and run DFS on the board — this prunes early when no prefix matches.

Real interview use-case: Implement trie, word search II, replace words, design search autocomplete, map sum pairs.

3.9 Graphs

What it is: A set of vertices (nodes) connected by edges. Can be directed/undirected, weighted/unweighted, cyclic/acyclic.

Internal mechanics:

```
Graph: 4 vertices, 4 edges (undirected)
  0 --- 1
  |   / |
```

```
  |  /  |
  | /   |
  2 --- 3

Adjacency List (preferred for sparse graphs):
  {
    0: [1, 2],
    1: [0, 2, 3],
    2: [0, 1, 3],
    3: [1, 2]
  }
  Space: O(V + E)
  Check edge (u,v): O(degree(u))
  Get neighbors: O(degree(u))

Adjacency Matrix (preferred for dense graphs):
      0  1  2  3
  0 [ 0  1  1  0 ]
  1 [ 1  0  1  1 ]
  2 [ 1  1  0  1 ]
  3 [ 0  1  1  0 ]
  Space: O(V²)
  Check edge (u,v): O(1)
  Get neighbors: O(V)

Weighted adjacency list:
  {0: [(1, 4), (2, 1)], ...}  ← (neighbor, weight)

Types:
  Undirected: edges go both ways
  Directed (Digraph): edges have direction
  DAG: Directed Acyclic Graph (used in topological sort)
  Tree: connected undirected graph with V-1 edges
```

Complexity:

Op	Adj List	Adj Matrix
Space	O(V+E)	$O(V^2)$
Add vertex	O(1)	$O(V^2)$
Add edge	O(1)	O(1)
Remove edge	O(E)	O(1)
Check edge (u,v)	O(degree)	O(1)
Get all neighbors	O(degree)	O(V)
BFS/DFS	O(V+E)	$O(V^2)$
Dijkstra	O((V+E)logV)	$O(V^2 \log V)$

When to use:

- Adj List: default, sparse graphs, BFS/DFS heavy
- Adj Matrix: dense graphs, need O(1) edge existence check, Floyd-Warshall

Key Python patterns:

```python
from collections import defaultdict, deque

# Build graph
graph = defaultdict(list)
for u, v in edges:
    graph[u].append(v)
    graph[v].append(u)  # undirected
```

```python
# BFS (shortest path in unweighted graph)
def bfs(graph, start, end):
    visited = {start}
    q = deque([(start, 0)])
    while q:
        node, dist = q.popleft()
        if node == end: return dist
        for nei in graph[node]:
            if nei not in visited:
                visited.add(nei)
                q.append((nei, dist + 1))
    return -1

# DFS (iterative)
def dfs(graph, start):
    visited = set()
    stack = [start]
    while stack:
        node = stack.pop()
        if node in visited: continue
        visited.add(node)
        for nei in graph[node]:
            stack.append(nei)

# DFS (recursive)
def dfs_rec(node, visited, graph):
    visited.add(node)
    for nei in graph[node]:
        if nei not in visited:
            dfs_rec(nei, visited, graph)

# Topological sort (Kahn's algorithm, BFS-based)
def topo_sort(n, edges):
    graph = defaultdict(list)
    in_degree = [0] * n
    for u, v in edges:
        graph[u].append(v)
        in_degree[v] += 1
    q = deque(i for i in range(n) if in_degree[i] == 0)
    order = []
    while q:
        node = q.popleft()
        order.append(node)
        for nei in graph[node]:
            in_degree[nei] -= 1
            if in_degree[nei] == 0:
                q.append(nei)
    return order if len(order) == n else []  # empty = cycle exists

# Dijkstra's (single source shortest path)
import heapq
def dijkstra(graph, start):
    dist = {node: float('inf') for node in graph}
    dist[start] = 0
    heap = [(0, start)]
    while heap:
        d, node = heapq.heappop(heap)
        if d > dist[node]: continue
        for nei, weight in graph[node]:
            new_d = d + weight
```

```
            if new_d < dist[nei]:
                dist[nei] = new_d
                heapq.heappush(heap, (new_d, nei))
    return dist
```

Interview takeaway: BFS for shortest path in unweighted graph. Dijkstra for weighted (non-negative). Detect cycle in directed graph: DFS with in-stack tracking. Topological sort only valid for DAGs. "Number of islands" = number of DFS/BFS calls on unvisited cells.

Real interview use-case: Number of islands, clone graph, course schedule, network delay time, Pacific Atlantic water flow, word ladder.

3.10 Union-Find / Disjoint Set

What it is: Data structure to track which elements belong to the same connected component. Supports two operations: `find` (which component?) and `union` (merge components).

Internal mechanics:

```
Initial state (6 elements, each its own component):
  parent = [0, 1, 2, 3, 4, 5]
  rank   = [0, 0, 0, 0, 0, 0]

Union(0, 1): find roots → 0,1; rank equal → make 1 child of 0
  parent = [0, 0, 2, 3, 4, 5]
  rank   = [1, 0, 0, 0, 0, 0]

Union(2, 3):
  parent = [0, 0, 2, 2, 4, 5]

Union(0, 2): find roots → 0, 2; rank[0]=1 > rank[2]=0 → 2 under 0
  parent = [0, 0, 0, 2, 4, 5]   ← 2's parent is now 0

Path Compression during find(3):
  find(3): parent[3]=2, parent[2]=0, parent[0]=0 → root=0
  Compress: set parent[3]=0, parent[2]=0 directly
  parent = [0, 0, 0, 0, 4, 5]   ← next find(3) is O(1)

After path compression and union by rank:
  find and union are effectively O(α(n)) ≈ O(1)
  where α = inverse Ackermann function
  α(n) < 5 for any practical n

Visual after several unions:
  Component 1: {0,1,2,3}  ← tree rooted at 0
  Component 2: {4}
  Component 3: {5}
       0
      /|\
     1 2 3   ← after path compression, flat tree
```

Complexity:

Op	Naive	Union by Rank	+ Path Compression
Find	O(n)	O(log n)	$O(\alpha(n)) \approx O(1)$
Union	O(n)	O(log n)	$O(\alpha(n)) \approx O(1)$
Space	O(n)	O(n)	O(n)

When to use:

- Connected components (dynamic)

- Detect cycles in undirected graph
- Kruskal's minimum spanning tree
- Grouping problems ("same group as")

Key Python patterns:

```python
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n
        self.components = n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])  # path compression
        return self.parent[x]

    def union(self, x, y):
        rx, ry = self.find(x), self.find(y)
        if rx == ry: return False  # already connected
        # Union by rank
        if self.rank[rx] < self.rank[ry]:
            rx, ry = ry, rx
        self.parent[ry] = rx
        if self.rank[rx] == self.rank[ry]:
            self.rank[rx] += 1
        self.components -= 1
        return True

    def connected(self, x, y):
        return self.find(x) == self.find(y)

# Example: count components in graph
def count_components(n, edges):
    uf = UnionFind(n)
    for u, v in edges:
        uf.union(u, v)
    return uf.components

# Detect cycle in undirected graph
def has_cycle(n, edges):
    uf = UnionFind(n)
    for u, v in edges:
        if not uf.union(u, v):  # already in same component
            return True
    return False

# Kruskal's MST
def kruskal(n, edges):
    uf = UnionFind(n)
    edges.sort(key=lambda e: e[2])  # sort by weight
    mst_cost = 0
    for u, v, w in edges:
        if uf.union(u, v):
            mst_cost += w
    return mst_cost
```

Interview takeaway: Always implement both path compression AND union by rank. Without both, you don't get the near-O(1) guarantee. Union-Find cannot efficiently handle edge deletion — if you need that, use Link-Cut Trees (not interview scope).

Real interview use-case: Number of provinces, redundant connection, accounts merge, satisfiability of equality equations.

3.11 LRU / LFU Cache Structures

What it is:

- **LRU (Least Recently Used)**: Evicts the item not accessed for the longest time. O(1) get and put.
- **LFU (Least Frequently Used)**: Evicts the item used fewest times (break ties by LRU). O(1) all operations.

Internal mechanics:

```
LRU Cache (capacity=3):
  Internals: HashMap + Doubly Linked List
  Map: key → node (O(1) access)
  DLL: most recent at front, least recent at tail

  get(1): move node 1 to front
  put(4): if full, remove tail (LRU), insert at front, update map

  State after put(1), put(2), put(3):
    [3] ⇔ [2] ⇔ [1] ← tail (LRU)
     ↑
    head (MRU)

  get(1): move 1 to head
    [1] ⇔ [3] ⇔ [2] ← tail (LRU)

  put(4): evict tail (2), add 4 at head
    [4] ⇔ [1] ⇔ [3]

LFU Cache (capacity=3):
  Internals: HashMap(key→(val,freq)) + HashMap(freq→DLL of keys)
             + min_freq tracker
  On access: increment freq, move to freq+1 list
  On evict: remove LRU from min_freq list
  Trickier: when min_freq list empties, min_freq++
```

Complexity:

Op	LRU	LFU
get	O(1)	O(1)
put	O(1)	O(1)

Key Python patterns:

Python

```python
from collections import OrderedDict

# LRU using OrderedDict (move_to_end trick)
class LRUCache:
    def __init__(self, capacity: int):
        self.cap = capacity
        self.cache = OrderedDict()  # key → value, ordered by access time

    def get(self, key: int) -> int:
        if key not in self.cache: return -1
        self.cache.move_to_end(key)  # mark as most recently used
        return self.cache[key]
```

```python

    def put(self, key: int, value: int) -> None:
        if key in self.cache:
            self.cache.move_to_end(key)
        self.cache[key] = value
        if len(self.cache) > self.cap:
            self.cache.popitem(last=False)  # remove LRU (first item)

# LRU from scratch (HashMap + DLL) — what interviewers actually want
class DLLNode:
    def __init__(self, key=0, val=0):
        self.key = key
        self.val = val
        self.prev = self.next = None

class LRUCacheManual:
    def __init__(self, capacity):
        self.cap = capacity
        self.cache = {}
        self.head = DLLNode()   # dummy head (MRU side)
        self.tail = DLLNode()   # dummy tail (LRU side)
        self.head.next = self.tail
        self.tail.prev = self.head

    def _remove(self, node):
        node.prev.next = node.next
        node.next.prev = node.prev

    def _insert_front(self, node):
        node.next = self.head.next
        node.prev = self.head
        self.head.next.prev = node
        self.head.next = node

    def get(self, key):
        if key not in self.cache: return -1
        node = self.cache[key]
        self._remove(node)
        self._insert_front(node)
        return node.val

    def put(self, key, val):
        if key in self.cache:
            self._remove(self.cache[key])
        node = DLLNode(key, val)
        self.cache[key] = node
        self._insert_front(node)
        if len(self.cache) > self.cap:
            lru = self.tail.prev
            self._remove(lru)
            del self.cache[lru.key]
```

Interview takeaway: LRU = HashMap + DLL with dummy head and tail. The dummy nodes eliminate all edge-case null checks. LFU is significantly harder — use HashMap(key→freq) + HashMap(freq→OrderedDict) + min_freq variable.

Real interview use-case: LRU cache (literal LeetCode 146), LFU cache (LeetCode 460), design Twitter, design browser history.

3.12 Segment Tree & Fenwick Tree (BIT)

What it is:

- **Segment Tree**: Tree where each node stores aggregate (sum/min/max) over a range. Supports range queries and point/range updates in O(log n).
- **Fenwick Tree (Binary Indexed Tree)**: Array-based structure using binary representation for prefix sums. Simpler to implement; less general.

Internal mechanics:

```
Array: [1, 3, 5, 7, 9, 11]  (0-indexed)

Segment Tree (sum):
                  [36: 0-5]
                 /           \
        [9: 0-2]              [27: 3-5]
        /       \             /        \
   [4: 0-1]  [5: 2-2]  [16: 3-4]  [11: 5-5]
    /    \               /     \
[1:0-0][3:1-1]      [7:3-3][9:4-4]

Query sum(1,4): decompose into [1-2] and [3-4] → 8 + 16 = 24
Update arr[2] = 10: update leaf, propagate to root → O(log n)
Space: O(4n) for array representation

Fenwick Tree (BIT) for prefix sums:
  Index:   1  2  3  4  5  6
  arr:    [1, 3, 5, 7, 9,11]
  BIT:    [1, 4, 5,16, 9,20]  ← each stores sum of specific range

  BIT[i] = sum of arr[i - lowbit(i) + 1 ... i]
  lowbit(i) = i & (-i)  ← isolates least significant bit

  Prefix sum query(5):
    i=5 (101): add BIT[5]=9, i=5-(5&-5)=4
    i=4 (100): add BIT[4]=16, i=4-(4&-4)=0 → stop
    Total: 9+16=25 ✓ (but wait: 1+3+5+7+9=25 ✓)

  Update(3, delta=2): propagate up
    i=3: BIT[3]+=2, i=3+(3&-3)=4
    i=4: BIT[4]+=2, i=4+(4&-4)=8 > n → stop
```

Complexity:

Op	Naive Array	Segment Tree	Fenwick Tree
Build	O(n)	O(n)	O(n log n)
Point update	O(1)	O(log n)	O(log n)
Range update	O(n)	O(log n)	O(log n)
Range query	O(n)	O(log n)	O(log n)
Space	O(n)	O(4n)	O(n)

When to use:

- Segment tree: range min/max queries, lazy propagation for range updates
- Fenwick tree: only prefix sums needed; cleaner code
- Both: dynamic range queries where array is mutated

Key Python patterns:

Python

```python
# Fenwick Tree (BIT) — prefix sum
class BIT:
    def __init__(self, n):
        self.n = n
        self.tree = [0] * (n + 1)  # 1-indexed

    def update(self, i, delta):    # add delta to position i
        while i <= self.n:
            self.tree[i] += delta
            i += i & (-i)          # move to parent

    def query(self, i):            # prefix sum [1..i]
        s = 0
        while i > 0:
            s += self.tree[i]
            i -= i & (-i)          # move to ancestor
        return s

    def range_query(self, l, r):   # sum [l..r]
        return self.query(r) - self.query(l - 1)

# Segment Tree (iterative, cleaner)
class SegmentTree:
    def __init__(self, nums):
        self.n = len(nums)
        self.tree = [0] * (2 * self.n)
        # Build
        for i, v in enumerate(nums):
            self.tree[self.n + i] = v
        for i in range(self.n - 1, 0, -1):
            self.tree[i] = self.tree[2*i] + self.tree[2*i+1]

    def update(self, i, val):
        i += self.n
        self.tree[i] = val
        while i > 1:
            i //= 2
            self.tree[i] = self.tree[2*i] + self.tree[2*i+1]

    def query(self, l, r):         # sum [l, r) half-open
        res = 0
        l += self.n; r += self.n
        while l < r:
            if l & 1: res += self.tree[l]; l += 1
            if r & 1: r -= 1; res += self.tree[r]
            l //= 2; r //= 2
        return res
```

Interview takeaway: Fenwick tree is simpler to code and sufficient for prefix sum + point update problems. Use segment tree when you need range updates or range min/max. The "count of smaller numbers after self" problem → BIT is the elegant solution.

Real interview use-case: Range sum query (mutable), count of smaller numbers after self, reverse pairs, falling squares.

3.13 Bloom Filter

What it is: A probabilistic, space-efficient data structure that answers "is X possibly in the set?" or "is X definitely NOT in the set?" — with false positives but no false negatives.

Internal mechanics:

```
Bloom Filter (m=20 bits, k=3 hash functions):

  Initial state:
  [0][0][0][0][0][0][0][0][0][0][0][0][0][0][0][0][0][0][0][0]

  Insert "cat" (hash1=3, hash2=7, hash3=15):
  [0][0][0][1][0][0][0][1][0][0][0][0][0][0][0][1][0][0][0][0]
                ↑               ↑               ↑

  Insert "dog" (hash1=3, hash2=10, hash3=17):
  [0][0][0][1][0][0][0][1][0][0][1][0][0][0][0][1][0][1][0][0]
  Note: bit 3 now set by both "cat" and "dog"!

  Query "cat": check bits 3,7,15 → all 1 → POSSIBLY in set ✓
  Query "hat": hash1=3, hash2=10, hash3=17 → all 1 → POSSIBLY in set × FALSE POSITIVE!
  Query "xyz": hash1=1 → 0 → DEFINITELY NOT in set ✓

  False positive probability ≈ (1 - e^(-kn/m))^k
  where n=items inserted, m=bit array size, k=hash functions

  Key: no false negatives (if bloom says NO, it's definitely NO)
       cannot delete items (counter-based bloom filters can)
       cannot enumerate stored elements
```

Complexity:

Op	Time	Space
Insert	O(k)	O(m)
Query	O(k)	O(1)
Delete	N/A	-

When to use:

- Cache pre-filtering: ”has this URL been seen before?” before hitting DB
- Distributed systems: ”does key exist on this node?” before network call
- Spell checkers: fast ”not a word” filter
- HBase/Cassandra: skip SSTable reads for non-existent keys

Ties to systems:

- Cassandra uses bloom filters to avoid unnecessary disk I/O
- Google Chrome used bloom filter for Safe Browsing URL blacklist
- Bitcoin uses bloom filters in SPV (Simplified Payment Verification)

Interview takeaway: Bloom filters trade accuracy for space and speed. False positives OK; false negatives are impossible. When an interviewer asks ”how would you check if a URL was seen before using O(1) space?” — bloom filter is the answer. Note: you can NOT delete from a standard bloom filter.

4 Choosing the Right Data Structure

Decision table: ”I need X → use Y”

Need	Use	Why
O(1) access by index	Array / List	Contiguous memory
O(1) key-value lookup	Hash Map	Hash function
Ordered key-value, range queries	BST / Sorted Map	In-order traversal

Need	Use	Why
Get min/max repeatedly	Heap	Root always min/max
Get kth smallest/largest	Heap of size k	O(n log k)
Prefix search / autocomplete	Trie	Shared prefix compression
LIFO (undo, recursion)	Stack	Push/pop from one end
FIFO (BFS, scheduling)	Queue / Deque	Enqueue/dequeue O(1)
Sliding window max/min	Monotonic Deque	Maintain monotone order
Next greater/smaller element	Monotonic Stack	Classic pattern
Connected components	Union-Find	Near-O(1) union/find
Shortest path (unweighted)	BFS (Queue)	Level-by-level expansion
Shortest path (weighted, non-neg)	Dijkstra (Heap)	Greedy + priority queue
Shortest path (negative weights)	Bellman-Ford	DP approach
All pairs shortest path	Floyd-Warshall	$O(V^3)$ DP
Range sum query + updates	Fenwick Tree / Segment Tree	O(log n) both
Range min/max query + updates	Segment Tree	BIT can't do min/max
O(1) LRU eviction	HashMap + DLL	O(1) access + O(1) order
Membership test, space-constrained	Bloom Filter	Probabilistic, tiny space
Count frequencies	HashMap / Counter	Hash map with integer values
Find duplicates	HashSet or sorting	O(n) with hash, O(n log n) sort
Merge k sorted sequences	Min-Heap of k elements	O(n log k)
Detect cycle (undirected graph)	Union-Find	Simpler than DFS coloring
Detect cycle (directed graph)	DFS with in-stack tracking	Union-Find doesn't work here
Topological ordering	Kahn's (BFS) or DFS	Works on DAGs only

5 Language Notes

5.1 Python

Need	Python	Notes
Dynamic array	`list`	`append` O(1) amortized
Hash map	`dict`	`defaultdict(list/int/set)`
Hash set	`set`	`add`, `in` O(1)
Ordered map (by key)	`sortedcontainers.SortedList`	Not stdlib; use `bisect` + list
Min heap	`heapq` (module)	Negate for max heap
Queue / Deque	`collections.deque`	`appendleft`, `popleft` O(1)
Counter	`collections.Counter`	`most_common(k)`
LRU built-in	`collections.OrderedDict`	`move_to_end`, `popitem`
Bisect (sorted insert)	`bisect.bisect_left/right`	Binary search on sorted list
Infinity	`float('inf')`	
Char to int	`ord('a')` → 97	`chr(97)` → 'a'

Python

```python
# Common Python idioms
from collections import defaultdict, Counter, deque, OrderedDict
import heapq
from bisect import bisect_left, bisect_right, insort

# Heap with custom key (push tuples)
heapq.heappush(heap, (priority, item))

# Sorted list simulation with bisect
import bisect
sorted_list = []
bisect.insort(sorted_list, 5)         # insert and maintain order
idx = bisect.bisect_left(sorted_list, 5)  # binary search

# defaultdict prevents KeyError
graph = defaultdict(list)
count = defaultdict(int)
```

5.2 Java

Need	Java Class	Notes
Dynamic array	`ArrayList<T>`	`add()` O(1) amortized
Hash map	`HashMap<K,V>`	Not ordered
Ordered map (by key)	`TreeMap<K,V>`	Red-Black tree, O(log n) ops
Ordered set	`TreeSet<T>`	Red-Black tree
Hash set	`HashSet<T>`	
Linked hash map	`LinkedHashMap<K,V>`	Insertion-order iteration (LRU)
Min/Max heap	`PriorityQueue<T>`	Min by default; pass `Comparator.reverseOrder()` for max
Stack	`Deque<T>` as stack	Prefer `ArrayDeque` over `Stack`
Queue	`ArrayDeque<T>`	Or `LinkedList<T>`
Deque	`ArrayDeque<T>`	

Java

```java
// Java idioms
PriorityQueue<Integer> minHeap = new PriorityQueue<>();
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());

Map<String, Integer> map = new HashMap<>();
map.getOrDefault(key, 0);
map.computeIfAbsent(key, k -> new ArrayList<>()).add(val);

Deque<Integer> stack = new ArrayDeque<>();  // use as stack
stack.push(1); stack.pop(); stack.peek();

Queue<Integer> queue = new ArrayDeque<>();  // use as queue
queue.offer(1); queue.poll(); queue.peek();

TreeMap<Integer, Integer> ordered = new TreeMap<>();
ordered.floorKey(x);    // largest key ≤ x
ordered.ceilingKey(x);  // smallest key ≥ x
```

5.3 C++

Need	C++ Container	Notes
Dynamic array	`vector<T>`	`push_back` O(1) amortized
Hash map	`unordered_map<K,V>`	O(1) avg
Ordered map (by key)	`map<K,V>`	Red-Black tree, O(log n)
Hash set	`unordered_set<T>`	
Ordered set	`set<T>`	+ `lower_bound`, `upper_bound`
Priority queue (max)	`priority_queue<T>`	Max by default
Priority queue (min)	`priority_queue<T,vector<T>,greater<T>>`	
Stack	`stack<T>`	
Queue	`queue<T>`	
Deque	`deque<T>`	

C++
```cpp
// C++ idioms
#include <bits/stdc++.h>
using namespace std;

unordered_map<int,int> freq;
freq[x]++;  // default-initializes to 0

// Min heap
priority_queue<int, vector<int>, greater<int>> minHeap;

// Ordered set with lower_bound
set<int> s;
auto it = s.lower_bound(x);  // O(log n)

// Sort with custom comparator
sort(v.begin(), v.end(), [](auto& a, auto& b){ return a[0] < b[0]; });
```

6 Real-Life Analogies

Data Structure	Analogy
Array	Numbered mailboxes in a row — access box 42 instantly
Hash Map	Library card catalog — look up by title, get shelf location
Linked List	Train cars — each car knows only the next car
Stack	Stack of plates — only access the top
Queue	Queue at a bank — first in, first served
Deque	Train that can board/exit from both ends
BST	Phone book — organized so you can binary search
Heap	Priority ER room — most critical patient always treated first
Trie	Autocomplete on your phone — shared prefix saves space
Graph	Road map — cities are nodes, roads are edges
Union-Find	Social groups — merging friend circles

Data Structure	Analogy
Segment Tree	Population pyramid — sum over any age range quickly
Fenwick Tree	Odometer with clever bit tricks for partial sums
LRU Cache	Small whiteboard — erase what was written longest ago for space
Bloom Filter	Bouncer list — might let someone in wrongly, never turns away regulars

7 Common Interview Questions

7.1 Arrays

- Two Sum (LeetCode 1)
- Best Time to Buy and Sell Stock (LC 121)
- Maximum Subarray / Kadane's (LC 53)
- Container With Most Water (LC 11)
- Trapping Rain Water (LC 42)
- Rotate Array (LC 189)
- Product of Array Except Self (LC 238)
- Merge Intervals (LC 56)

7.2 Strings

- Longest Substring Without Repeating Characters (LC 3)
- Minimum Window Substring (LC 76)
- Group Anagrams (LC 49)
- Valid Palindrome (LC 125)
- Encode and Decode Strings (LC 271)
- Longest Palindromic Substring (LC 5)
- String to Integer (atoi) (LC 8)

7.3 Hash Maps/Sets

- Two Sum (LC 1)
- Top K Frequent Elements (LC 347)
- Longest Consecutive Sequence (LC 128)
- Subarray Sum Equals K (LC 560)
- Valid Sudoku (LC 36)
- Find All Anagrams in a String (LC 438)

7.4 Linked Lists

- Reverse Linked List (LC 206)
- Merge Two Sorted Lists (LC 21)
- Linked List Cycle (LC 141)
- Reorder List (LC 143)
- Remove Nth Node From End (LC 19)
- Merge K Sorted Lists (LC 23)

7.5 Stacks/Queues

- Valid Parentheses (LC 20)
- Min Stack (LC 155)

- Daily Temperatures (LC 739)
- Largest Rectangle in Histogram (LC 84)
- Sliding Window Maximum (LC 239)
- Evaluate Reverse Polish Notation (LC 150)

7.6 Trees

- Maximum Depth of Binary Tree (LC 104)
- Invert Binary Tree (LC 226)
- Same Tree (LC 100)
- Binary Tree Level Order Traversal (LC 102)
- Validate Binary Search Tree (LC 98)
- Lowest Common Ancestor of BST (LC 235)
- Kth Smallest Element in BST (LC 230)
- Serialize and Deserialize Binary Tree (LC 297)
- Diameter of Binary Tree (LC 543)

7.7 Heaps

- Kth Largest Element in an Array (LC 215)
- K Closest Points to Origin (LC 973)
- Find Median from Data Stream (LC 295)
- Task Scheduler (LC 621)
- Merge K Sorted Lists (LC 23)

7.8 Tries

- Implement Trie (LC 208)
- Word Search II (LC 212)
- Replace Words (LC 648)
- Design Search Autocomplete System (LC 642)

7.9 Graphs

- Number of Islands (LC 200)
- Clone Graph (LC 133)
- Pacific Atlantic Water Flow (LC 417)
- Course Schedule (LC 207) — cycle detection
- Number of Connected Components (LC 323)
- Word Ladder (LC 127) — BFS shortest path
- Network Delay Time (LC 743) — Dijkstra
- Cheapest Flights Within K Stops (LC 787) — modified Dijkstra/Bellman-Ford

7.10 Union-Find

- Number of Provinces (LC 547)
- Redundant Connection (LC 684)
- Accounts Merge (LC 721)
- Satisfiability of Equality Equations (LC 990)

7.11 LRU/LFU

- LRU Cache (LC 146)
- LFU Cache (LC 460)

7.12 Segment Tree / BIT

- Range Sum Query — Mutable (LC 307)
- Count of Smaller Numbers After Self (LC 315)
- Reverse Pairs (LC 493)

8 Typical Mistakes Candidates Make

1. **Using `list.pop(0)` for queues** — It's O(n). Always use `collections.deque` and `popleft()`.
2. **String concatenation in loops** — `result += s` inside a loop is O(n^2). Use `"".join(parts)`.
3. **Not handling the empty/single-element edge case** — For linked lists, trees, arrays. Check `if not root`, `if not nums`, `if len(nums) < 2` early.
4. **BST validation using only local `left < root < right`** — This misses nodes in subtrees. Pass bounds `(lo, hi)` down the recursion.
5. **Modifying a collection while iterating it** — Causes RuntimeError. Copy or collect indices first.
6. **Forgetting to mark nodes visited before enqueuing (BFS)** — Leads to exponential revisits. Mark visited when you enqueue, not when you dequeue.
7. **Using a set as a hash map key** — Sets are unhashable. Use `frozenset` or `tuple(sorted(...))`.
8. **Forgetting Python's `heapq` is min-heap only** — For max-heap, push `-val` and negate on pop.
9. **O(n^2) nested loop when O(n) hash map exists** — When you write two nested loops looking for a pair, always ask: "can I precompute what I'm looking for?"
10. **Not using dummy head node for linked lists** — Head deletion becomes a special case. Dummy head eliminates it.
11. **Recursion without memoization** — Fibonacci, path counting, etc. naturally revisit sub-problems. Add `@lru_cache` or a memo dict.
12. **Union-Find without path compression** — Without it, find is O(n). Always compress.
13. **Assuming `dict` maintains insertion order in Python 2** — It doesn't. Python 3.7+ guarantees it; don't rely on this in cross-version code.
14. **Off-by-one in sliding window** — When window is `[l, r]`, size is `r - l + 1`. Sketch a small example before coding.
15. **Passing arrays by reference and mutating them** — In Python lists are passed by reference. If you need to preserve the original, copy with `arr[:]`.

9 Revision Cheat Sheet

9.1 Complexities to Memorize (Absolute Must)

```
Array access:     O(1)     | Hash map get/put: O(1) avg
Array search:     O(n)     | BST search:       O(log n) balanced
Append to list:   O(1) A   | Heap push/pop:    O(log n)
List insert/del:  O(n)     | Build heap:       O(n)  ← non-obvious!
Trie ops:         O(m)     | Segment tree:     O(log n) query/update
BFS/DFS:          O(V+E)   | Union-Find:       O(α(n)) ≈ O(1)
Dijkstra:         O((V+E)log V)
Heapsort:         O(n log n)
Sort:             O(n log n)
Binary search:    O(log n)
String join:      O(n)     | String concat loop: O(n²) ← DANGER

A = amortized
```

9.2 Decision Hooks (6 questions to ask every interview)

1. **"Do I need O(1) lookup?"** → Hash map or hash set.
2. **"Do I need sorted order or range queries?"** → BST, sorted array + binary search, or segment tree.
3. **"Do I need repeated min/max?"** → Heap.
4. **"Is this a traversal/reachability problem?"** → BFS (shortest path) or DFS (connectivity, cycles).
5. **"Am I connecting/grouping things?"** → Union-Find.
6. **"Do I need prefix operations?"** → Trie (strings) or Fenwick tree (numbers).

9.3 Quick Pattern Mapping

Pattern	Structure
Sliding window	Array + two pointers
Sliding window max/min	Monotonic deque
Next greater element	Monotonic stack
Level-order traversal	Queue (BFS)
Path/exhaustive search	Stack (DFS) / recursion
Shortest path (unweighted)	BFS
Shortest path (weighted)	Dijkstra (min-heap)
Connected components	BFS/DFS or Union-Find
Top K elements	Heap of size K
Frequency counting	Hash map / Counter
Deduplication	Hash set
Prefix sum	Fenwick tree / prefix array
Range sum (mutable)	Fenwick tree
Range min/max (mutable)	Segment tree
Word prefix search	Trie
Order + O(1) eviction	LRU (HashMap + DLL)
Large-scale membership	Bloom filter

9.4 The 5 Structures You MUST Be Able to Code From Scratch

1. **Hash Map** (open addressing or chaining — at least explain internals)
2. **Linked List** with insert, delete, reverse
3. **Binary Heap** with push, pop, heapify
4. **Trie** with insert, search, startsWith
5. **LRU Cache** (HashMap + doubly linked list)

9.5 Memory Aids

- **Heap is NOT a BST** — heap has parent $\leq$ children (only), BST has left < parent < right.
- **Trie depth = key length, not n** — n = number of keys.
- **Union-Find has no delete** — it's append-only by design.
- **BFS finds shortest path; DFS does not** — in unweighted graphs.
- **`heapify` is O(n)** — this surprises everyone. Proof: most nodes are at the bottom and need to sift down only a little.
- **Hash map worst case O(n)** — when all keys hash to same bucket (adversarial input or bad hash). In practice O(1).

- **AVL is stricter than Red-Black** — AVL: max height diff 1; RB: height $\leq$ 2*log(n). RB faster for insert/delete, AVL faster for lookup.